PREQUAL-RS

Reproduction of **"Load is not what you should balance: Introducing Prequal"**, Wydrowski, Kleinberg, Rumble, Archer — NSDI 2024 (`bib/nsdi24-wydrowski.pdf`).

This document covers: (1) what the paper proposes, (2) how this repo implements it, (3) the correctness review — two real bugs found and fixed, (4) why the testbed needed per-replica CPU isolation and how the experiments are set up, (5) the results from the CI experiment run and how they compare with the paper's findings, and (6) two questions of our own that the reproduction ended up answering.

---

1. What the paper proposes

Google's production load balancer (WRR) balances **CPU utilization** across replicas. The paper argues this is the wrong goal: CPU utilization is a *trailing* signal (it only means something when averaged over a window), and replicas differ widely in their *spare capacity* because machines are shared with unpredictable **antagonist** processes. A balancer can equalize CPU perfectly and still send queries to replicas with no headroom — which is exactly what causes tail latency and errors.

Prequal (PRobing to REduce QUeuing And Latency) instead picks replicas using two *real-time* signals obtained by actively probing:

- **RIF (requests in flight)** — an instantaneous counter, a *leading* indicator of future load, and a bound on per-replica RAM;
- **latency estimate** — the median of recently completed queries at (or near) the replica's current RIF.

Key design elements:

- **Asynchronous probing.** Each query triggers `r_probe` probes (default 3) to replicas sampled uniformly *without replacement* the responses land in a bounded **probe pool** (default size 16) and are used by *later* queries, so probing stays off the critical serving path.
- **HCL (hot-cold lexicographic) selection.** Clients estimate the RIF distribution across replicas from pooled probes; a probe is **hot** if its RIF is above the `Q_RIF` quantile (default `2^-0.25 ≈ 0.84`), otherwise **cold**. If all probes are hot, pick the one with the lowest RIF; otherwise pick the **cold probe with the lowest latency**. `Q_RIF = 0` reduces to RIF-only control, `Q_RIF = 1` to latency-only control. If the pool holds fewer than 2 probes, fall back to a uniformly random replica.
- **Pool management** against *staleness*, *depletion*, and *degradation*:
  - probes expire after a TTL (1 s) and after `b_reuse` uses, where `b_reuse = max(1, (1+δ) / ((1−m/n)·r_probe − r_remove))` (the paper's reuse-budget formula);
  - the oldest probe is evicted when a new one would overflow the pool;
  - `r_remove` probes (default 1) are deleted per query, **alternating** between the oldest probe and the *worst* probe (highest RIF if any probe is hot, else highest latency) — without this, selection bias would leave the pool full of bad probes;
  - a client makes up for its own dispatches by incrementing the RIF on the pooled probes of the replica it just picked.

On YouTube, switching from WRR to Prequal cut tail latency by 40–50 %, cut tail RIF 5–10×, and nearly eliminated load-imbalance errors. On the paper's 100-client / 100-server testbed, Prequal survives loads up to 1.74× the CPU allocation with bounded latency and **zero errors** while WRR collapses just past allocation; it beats every other selection rule the paper tests, is insensitive to the probing rate down to about one probe per query, and works best with `Q_RIF` roughly between 0.6 and 0.9 — pure latency control (`Q_RIF = 1`) is much worse than adding even a little RIF aversion.

---

2. How this repo implements it

One binary, two subcommands (`src/cli.rs`); the client and every replica are independent processes speaking plain HTTP, so the system deploys unchanged on a real multi-host cluster — only the `--servers` URL list changes. The experiment testbed runs each replica in its own Docker container with a hard CPU limit.

Piece	File
Replica: `/work` (iterated-hash CPU work), `/probe` (RIF + latency + CPU)	`src/servers/replica.rs`
RIF-indexed latency estimator (ring of recent completions, median near current RIF, freshness window)	`src/servers/replica.rs` `LatencyRing`
Antagonist: square-wave CPU burner with phase offset, burning inside its replica's allocation	`src/servers/antagonist.rs`
Probe pool: TTL, reuse budget, oldest-eviction, alternating oldest/worst removal, RIF self-compensation	`src/client/pool.rs`
HCL selection + hot threshold (per-replica-deduplicated RIF quantile)	`src/client/pool.rs` `hcl_select`
Policies: `prequal`, `random`, `round-robin`, `po2` (sync-probe power-of-two by RIF), `wrr` (weights ∝ 1/CPU-util)	`src/client/policy.rs`
Open-loop Poisson load generator, per-query work Normal(mean, sd = mean) truncated at 0	`src/client/mod.rs`
Latency histograms (HdrHistogram), per-replica counts, error counts	`src/metrics/collector.rs`
Testbed: CPU-capped replica containers, self-calibrating load levels	`experiments/run.sh`, `experiments/Dockerfile.runtime`

Deliberate scale-downs vs. the paper's testbed (100 clients / 100 servers with enforced 10 % CPU allocations): N replica containers (6 locally, 4 on the CI runner) plus one client process, with `--balancers 6` independent probe pools standing in for many client replicas (a single shared pool would herd every query onto the same "best" replica), and a pool capacity of 4 (< n, so each pool sees a random subset of replicas — the same decorrelation the paper gets from a pool much smaller than the fleet).

Details that follow the paper exactly: probe targets sampled without replacement; hot/cold split at the `Q_RIF` quantile with RIF-only and latency-only as the `0`/`1` endpoints; fallback to uniform random below pool occupancy 2; removal alternating oldest/worst, with "worst" defined as the paper defines it; oldest-eviction on overflow; RIF compensation for the client's own dispatches; RIF spans arrival to completion, so it counts queued queries too.

Small documented deviations: the latency estimator also rescales by `(rif+1)/(rif_at_probe+1)` (a processor-sharing correction; the paper mentions this kind of compensation as a nice-to-have); WRR weights use `1/cpu_util` rather than `qps/cpu_util` (equivalent when WRR keeps per-replica QPS roughly equal, which it does); probe timeout is 100 ms rather than 3 ms; `r_probe` is an integer, so the fractional probing rates the paper sweeps below 1 can't be reproduced.

---

3. Correctness review — what was wrong, what was fixed

3.1 Bug: reuse-budget formula grouped wrongly (fixed)

The paper's reuse-budget formula:

```
b_reuse = max{1, (1+δ) / ((1−m/n)·r_probe − r_remove)}
```

The denominator is a **net pool-growth rate**: probes *grow* the pool at rate `(1−m/n)·r_probe` per query (a probe of a replica already pooled doesn't grow it), and removals shrink it at `r_remove` per query. `src/config.rs` computed

```
let net = (1.0 - m / n) * (self.r_probe as f64 - self.r_remove as f64);  // WRONG
```

i.e. `(1−m/n)·(r_probe − r_remove)` — the `(1−m/n)` factor was applied to the removal rate too. With the paper's own defaults (m=16, n=100, r_probe=3, r_remove=1) both versions happen to round to `b_reuse = 2`, which is probably why the mistake went unnoticed; but the formulas diverge elsewhere — e.g. at m=48, n=100 the wrong formula gives `ceil(2/1.04) = 2` while the correct one gives `ceil(2/0.56) = 4`, and whenever `(1−m/n)·r_probe ≤ r_remove` the correct formula says no finite reuse budget can keep the pool from draining (the code's `u32::MAX` branch) while the wrong one still returned finite budgets. **Fixed** to `(1.0 - m/n) * r_probe - r_remove`, with unit tests pinning the paper-default value (`b_reuse = 2`) and the degenerate cases.

One consequence at testbed scale: with pool m=4 and n=6 replicas, `(1−4/6)·3 − 1 = 0`, so the corrected formula says reuse must be unbounded — probes then retire only via TTL, per-query removal, and eviction. That is the honest reading of the paper's model when `m/n` is large.

3.2 Bug: RIF leaks when a query is cancelled (fixed)

The `/work` handler incremented the RIF counter at arrival and decremented it after the work — as two separate statements. When a client times out, the HTTP connection closes and axum **drops the handler future**, so the decrement never ran: every timed-out query permanently inflated the replica's advertised RIF. Under overload with 5 s timeouts this leaked thousands of counts, corrupting the primary load signal exactly when it matters most (and skewing the RIF-indexed latency estimator, which scales with the live RIF). **Fixed** with an RAII guard (`RifGuard`) whose `Drop` decrements the counter, so cancellation and completion are both counted out.

3.3 Testbed flaw: no per-replica CPU isolation (fixed)

Not a bug in Prequal's logic, but the reason early experiment attempts could not reproduce the paper: replicas ran as bare processes sharing one host scheduler, with unbounded `spawn_blocking` concurrency. The consequences, observed before the fix:

- all policies produced statistically identical latency below saturation — with shared cores there is no per-replica capacity for a balancer to route around, and the OS scheduler is already doing the "balancing";
- an antagonist attached to one replica stole cycles from *every* replica, so avoiding the "hot" replica avoided nothing;
- above saturation, hundreds of concurrently spinning threads produced machine-wide, non-monotonic collapse (a policy could look fine at 1.15× yet melt at 1.05×).

The fix has two layers. In-process, `--cpu-alloc` now bounds concurrent work execution via semaphore worker slots (queries queue beyond it, which is what makes RIF a leading indicator), and the antagonist burns *inside* its own replica's allocation with deficit tracking, so a spike steals that replica's capacity specifically. At the testbed level, each replica runs in its own Docker container with a hard `--cpus` limit — the kernel-enforced equivalent of the paper's per-VM guaranteed CPU allocation. The system code itself stays deployment-agnostic (plain HTTP between processes).

3.4 Verified correct against the paper

- **HCL rule** (`hcl_select`): cold = RIF ≤ threshold at the `Q_RIF` quantile of the *deduplicated, latest-per-replica* RIF distribution — the paper's estimate of the RIF distribution across replicas; lowest-latency cold probe, else lowest-RIF. Unit-tested, including the `Q_RIF = 0` ⟹ RIF-only and `Q_RIF = 1` ⟹ latency-only endpoints.
- **Removal machinery**: alternation oldest/worst, worst = highest RIF if any hot else highest latency; per-query `r_remove`; TTL; reuse budget; oldest-eviction on overflow. All unit-tested (10 tests, all passing).
- **Random fallback below pool occupancy 2**, as the paper recommends.
- **Probing**: `r_probe` async probes per query to distinct uniform targets; responses enter the pool off the critical path.
- **Defaults** match the paper's baseline configuration: pool 16 (library default), TTL 1 s, $\delta = 1$, `Q_RIF = 2^-0.25 ≈ 0.84`, `r_probe = 3`, `r_remove = 1`.

- **Workload** matches the paper's: iterated-hash CPU work, per-query cost Normal(mean, sd = mean) truncated at zero, open-loop Poisson arrivals; the arrival scheduler uses an absolute timeline with catch-up so the offered rate is honored even though per-sleep granularity is 1 ms.
- **Sanity checks** after the fixes: with fresh isolated replicas at 90 % load, `prequal` (baseline, RIF-only, and single-balancer variants) and sync-probe `po2` all landed within a few percent of each other with zero errors — no policy-specific oddity.

Minor fixes alongside: added `--work-factor` to the server so the fast/slow experiment (work inflated 2× on half the replicas, standing in for older hardware) can be reproduced; comment typos; one clippy warning.

3.5 A note on measurement hygiene

Two things silently ruined early runs and are now handled by the harness: (a) back-to-back runs bleed into each other (queued work, thermal throttling on laptops), so the suite inserts drain/cooldown gaps; (b) a hardcoded "capacity" is wrong on any other machine, so the suite *self-calibrates* — it walks the random policy up a ×1.2 load ladder until p50 exceeds 4× the unloaded p50 (or errors appear) and expresses every experiment's load as a fraction of that measured knee.

---

4. Experiments

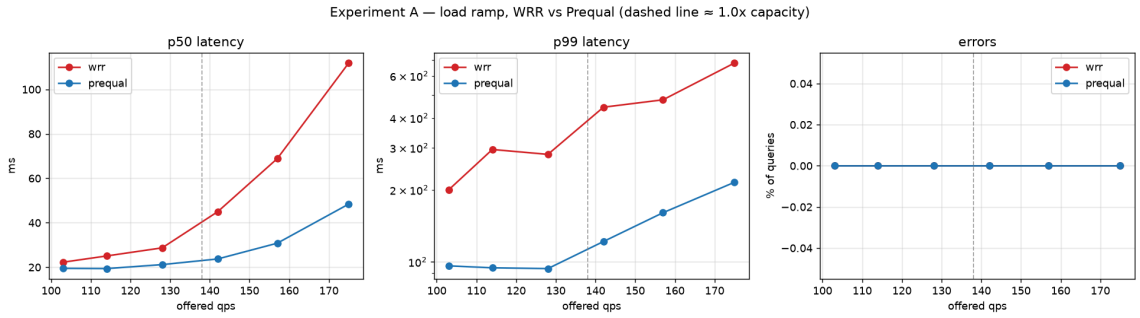
Run on GitHub Actions (`.github/workflows/experiments.yml`): a 4-vCPU `ubuntu-latest` runner hosting 4 replica containers at `--cpus 0.75` each plus the client process. Mean work 6 M hash iterations, 5 s query timeout (as in the paper), 40 s per data point, `--balancers 6`, cooldowns between runs. Raw JSON, tables and figures are uploaded as the `experiment-results` artifact of each run; `experiments/run.sh` reproduces locally (Docker required, `USE_DOCKER=0` falls back to bare processes).

- **A. Load ramp.** Every replica runs a staggered square-wave antagonist (60 % of its allocation, 20 s period) — the paper's unpredictable time-varying antagonist load. WRR vs Prequal at 0.75×, 0.83×, 0.93×, 1.03×, 1.14×, 1.27× of measured capacity (stepping up by roughly 10 % each time, as the paper does).
- **B. Replica-selection rules.** random, round-robin, WRR, Po2C (synchronous probes, lower RIF wins), Prequal, at 70 % and 90 % of capacity on the antagonist testbed.
- **C. `Q_RIF` sweep.** No antagonists; replicas split half fast / half slow (`--work-factor 2`); `Q_RIF ∈ {0, 0.35, 0.59, 0.84, 0.97, 1.0}` at 90 % of that testbed's capacity.
- **D. Probing rate.** `r_probe ∈ {1, 2, 3, 4}` at 1.15× capacity (the paper runs this experiment very hot).

5. Results

From GitHub Actions run 29167802521 (artifact `experiment-results`, copied to `experiments/ci-artifact/`). Calibrated capacity: **138 qps** on the antagonist testbed, **115 qps** on the fast/slow testbed (4 replicas × 0.75 CPU; one query ≈ 17 ms unloaded on a runner core). All numbers are client-observed end-to-end latency in ms.

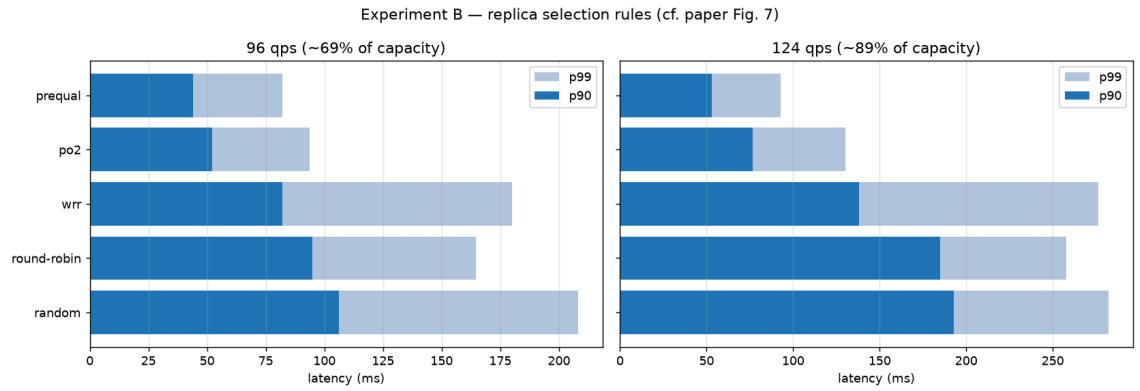
5.1 Experiment A — load ramp, WRR vs Prequal



offered	×capacity	Prequal p50	Prequal p99	WRR p50	WRR p99
103	0.75	19.5	96	22.3	200
114	0.83	19.4	94	25.1	295
128	0.93	21.2	94	28.7	281
142	1.03	23.7	122	45.0	443
157	1.14	30.8	160	68.9	476
175	1.27	48.4	215	111.9	678

Both policies return **zero errors** throughout. This is the paper's headline result in miniature: below allocation the two are close at the median, but as soon as load crosses the allocation line WRR's latency runs away — its p50 grows 5× and its p99 crosses the half-second mark — while Prequal degrades gracefully (p99 still 215 ms at 1.27×, a 3.2× advantage over WRR). WRR's trailing, time-averaged CPU signal keeps sending queries to replicas whose antagonist just spiked; Prequal's instantaneous RIF + latency signals route around each spike as it happens. This matches the paper's findings; our milder collapse reflects the smaller fleet and the 5 s timeout never being reached.

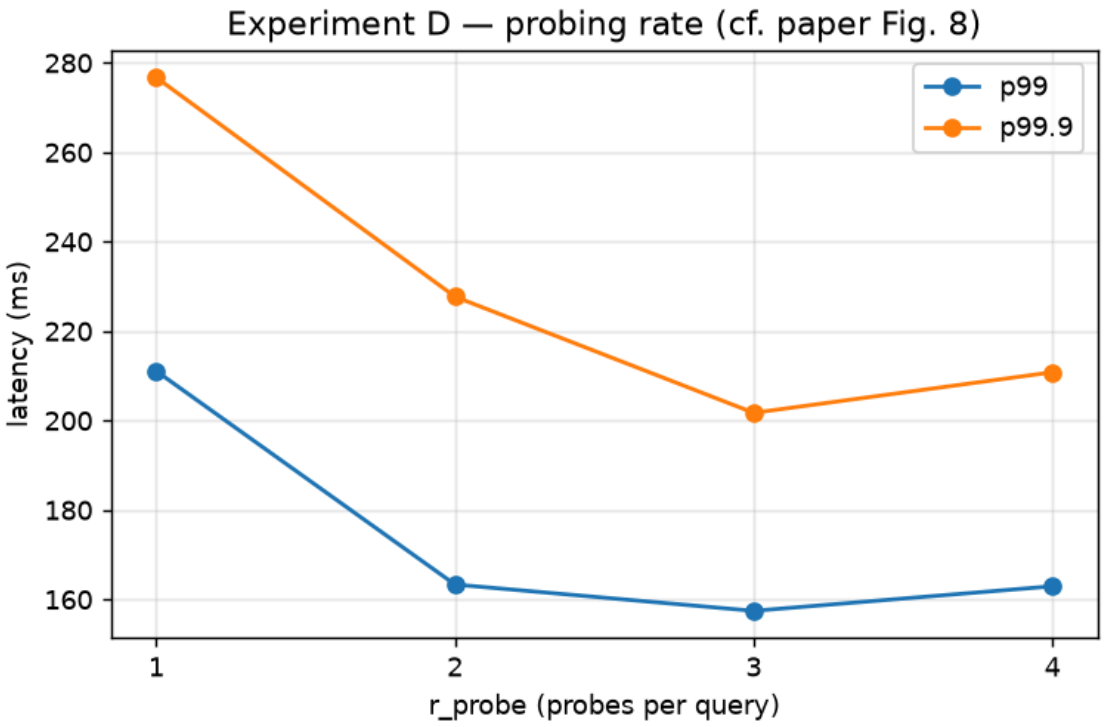
5.2 Experiment B — replica selection rules



policy	p50 @70%	p99 @70%	p50 @90%	p99 @90%
random	24.1	208	43.4	282
round-robin	23.8	165	46.4	258
wrr	21.2	180	27.7	277
po2 (sync, by RIF)	18.6	94	24.0	130
prequal	**17.5**	**82**	**21.4**	**93**

The ordering matches the paper's results: Prequal best at every quantile and both load levels, probing-based po2 a clear second, and the probe-less policies (WRR, round-robin, random) far behind, with the gap widening at 90 % load (Prequal's p99 advantage over WRR grows from 2.2× to 3.0×). Note that our po2 is a *strong* baseline — synchronous, perfectly fresh server-side RIF at negligible probe cost on a local network — yet Prequal still beats it at the tail (93 vs 130 ms p99 at 90 %) while keeping probing off the critical path, the paper's central design argument.

5.3 Experiment D — probing rate

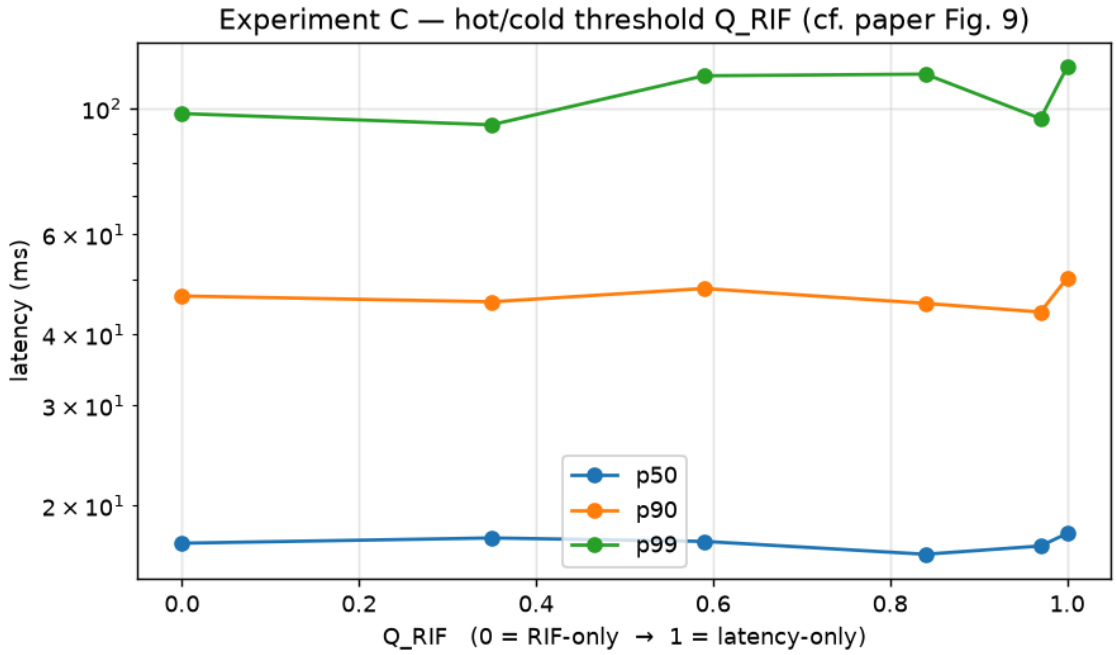


r_probe	p50	p90	p99	p99.9
1	42.3	134	211	277
2	31.6	90	163	228

3	35.0	92	157	202
4	32.0	88	163	211

At 1.15× capacity, performance is flat across `r_probe ∈ {2, 3, 4}` and degrades visibly only at 1 probe/query (p50 +30 %, p99 +34 % vs `r_probe=3`). This matches the paper's take-home point that Prequal is fairly insensitive to the probing rate until it drops below one probe per query — the default of 3 sits comfortably on the flat part of the curve. (Fractional rates below 1, where the paper sees serious degradation, aren't reachable: `r_probe` is an integer here.)

5.4 Experiment C — Q_RIF sweep on fast/slow replicas



Q_RIF	p50	p99	p99.9	fast:slow traffic
0.00 (RIF-only)	17.2	98	145	2.07
0.35	17.5	94	143	2.27
0.59	17.3	114	179	2.76
0.84	16.4	115	178	2.94
0.97	17.0	96	161	2.79
1.00 (latency-only)	17.9	118	215	3.04

Two of the paper's observations reproduce cleanly. First, the traffic split: as `Q_RIF` rises the balancer routes ever more queries to the fast replicas (2.07:1 ⟶ 3.04:1) — as in the paper, since latency-based control naturally favors fast machines. Second, the endpoint behavior: pure latency

control (`Q_RIF = 1`) has the worst tail (p99.9 215 ms, 50 % above RIF-only), echoing the paper's finding that ignoring RIF entirely is a bad idea, while RIF-only control works fine. The interior sweet spot the paper finds (`Q_RIF` between the endpoints beating RIF-only) does *not* show at this scale — with only 4 replicas and a pool of 4, the RIF quantile estimate is too coarse for the hot/cold distinction to add value over RIF-only control; all settings except latency-only are within noise of each other at 90 % load with zero errors.

5.5 Summary

- Prequal **works**: it beats every baseline at the tail, degrades gracefully to 1.27× of allocation with zero errors, and routes around antagonist spikes that cripple CPU-balancing — the paper's core claims.
- The probing-rate insensitivity (≥ 2 probes/query) and the failure of pure latency control both reproduce.
- The one paper result that needs more scale than 4 replicas is the interior `Q_RIF` optimum — at this size, RIF-dominant settings are simply tied.

---

6. Beyond the paper — two questions of our own

Reproducing the experiments ended up answering two questions the paper does not ask. Both started out as problems — one as a confusing sweep result, one as a testbed that refused to show any policy differences — and both turned out to be findings about *when* Prequal's machinery actually matters. They are negative results, but the useful kind: they mark the boundaries of the paper's claims.

6.1 Does the hot/cold distinction still add value on a small fleet?

The paper tunes `Q_RIF` on a fleet of 100 replicas and finds that a mid-range setting beats both endpoints. Our sweep (experiment C) asks the same question at the opposite end of the scale: 4 replicas, probe pool of 4.

The answer is no. Every setting that respects RIF at all — from `Q_RIF = 0` (RIF-only) up to 0.97 — lands within noise of the others at 90 % load with zero errors; only pure latency control (`Q_RIF = 1`) stands out, and only by being worse (p99.9 about 50 % above RIF-only). The reason is structural, not statistical bad luck: the hot threshold is a quantile of the RIF distribution estimated from the pool, and with at most 4 distinct replicas in the pool that distribution has only a handful of possible values. The quantile can barely move, so the hot/cold split adds almost no information on top of "pick the lowest RIF".

The practical takeaway: on a small fleet, the simplest configuration (RIF-only, `Q_RIF = 0`) is all you need — the quantile machinery is a large-fleet refinement, not a requirement. And it is safe to leave the default in place: mid-range settings don't win here, but they don't lose either. The only real mistake at any scale is ignoring RIF completely.

6.2 Does Prequal help when replicas share cores without capacity

isolation? The isolation story of section 3.3 was found while debugging, but it answers a genuine deployment question: many small setups co-locate all service processes on one machine, with no pinned CPU allocations. Does Prequal buy anything there?

Again no — and not because of a bug. With replicas running as bare processes on shared cores we measured, repeatedly: all policies statistically identical below saturation (random included); an antagonist attached to one replica slowing down *all* of them, so routing around the "hot" one avoided nothing; and non-monotonic collapse above saturation. The explanation is simple: load balancing can only help when replicas have *private* capacity, so that one can be busy while another has headroom. When every replica draws from the same shared cores, the OS scheduler has already smeared the load evenly, and RIF and latency look the same everywhere — there is nothing left for the balancer to see or to route around.

The takeaway is that the paper's per-VM guaranteed CPU allocation is not a testbed detail; it is a load-bearing assumption. Prequal (and by the same logic any load-signal-based balancer) needs replicas whose capacity is isolated enough that their load signals can actually differ. Deploying it over co-located, unisolated processes costs the probing overhead and returns nothing.

---

7. Reproducing

```
cargo build --release
cargo test --release            # 10 unit tests on pool/HCL/config
experiments/run.sh              # Docker testbed, self-calibrating (~35 min)
uv run experiments/analyze.py   # tables on stdout + experiments/figures/*.png
```

On GitHub: the `experiments` workflow runs on every push to `dev` touching `src/` or `experiments/` (or manually via *Run workflow*); results appear in the run's step summary and as the `experiment-results` artifact.